

Name (Last, First): _____

This practice exam covers the exercises you have done so far, 00r_hello_rust through 12c_wc. It consists of 5 questions on 21 pages; be sure you have the entire exam before starting. The point value of each question is indicated at its beginning; the entire exam has 100 points. Individual parts of a multi-part question are generally assigned approximately the same point value: exceptions are noted.

This exam is closed book. The one reference you may use is the printed course Cheatsheet. No electronic devices of any kind. You may not share material with classmates during the exam.

Be concise and clearly indicate your answer. Presentation and simplicity of your answers may affect your grade. Answer each question in the space following the question. If you find it necessary to continue an answer elsewhere, indicate clearly the location of its continuation and label its continuation with the question number and subpart if appropriate.

You should read through all the questions first, then pace yourself.

If you need to ask a question during the exam, please raise your hand and I will come to your desk. Do not come to me.

The questions begin on the next page.

Problem	Possible	Score
1	20	
2	20	
3	20	
4	20	
5	20	
Total	100	

1. (_____/20 points)

Short Questions

(a) (5 points) Exercise 01r rounds addresses up to page boundaries with these functions:

```
pub const PAGE_SIZE: usize = 0x1000;

pub fn pgroundup(addr: usize) -> usize {
    (addr + PAGE_SIZE - 1) / PAGE_SIZE * PAGE_SIZE
}

pub fn checked_pgroundup(addr: usize) -> Option<usize> {
    match addr.checked_add(PAGE_SIZE - 1) {
        Some(bumped) =>
            Some(bumped / PAGE_SIZE * PAGE_SIZE),
        None => None,
    }
}

pub fn count_pages(start: usize,
                    end: usize) -> usize {
    let mut addr = pgroundup(start);
    let mut pages = 0;
    loop {
        if addr + PAGE_SIZE > end {
            break pages;
        }
        addr += PAGE_SIZE;
        pages += 1;
    }
}
```

Give the value of each expression. Show your work.

- (i) `pgroundup(0x8000_2001)`
- (ii) `pgroundup(0x8000_5000)`
- (iii) `count_pages(0x8000_0800, 0x8000_3800)`
- (iv) `pgroundup(usize::MAX - 5)`. What happens in a debug build, and what value does it return in a release build? What does `checked_pgroundup(usize::MAX - 5)` return instead?

(b) (5 points) In exercise 04r, `Pte` derives `Copy` and `PageGuard` implements `Drop`:

```
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub struct Pte(pub u64);          // pa(self) and flags(self) take self BY VALUE

pub struct PageGuard<'a> {
    free: &'a mut Vec<usize>,
    pa: usize,
}
// PageGuard::take(free: &'a mut Vec<usize>) -> PageGuard<'a>
// pa(&self) -> usize    entry(&self, flags: u64) -> Pte    release(self) {}
// PTE_V and PTE_R are u64 flag bits, 1 and 2.

impl Drop for PageGuard<'_> {
    fn drop(&mut self) {
        self.free.push(self.pa);
    }
}
```

Consider this code:

```
let mut free = vec![0x8000_0000, 0x8000_1000];
let page = PageGuard::take(&mut free);    // 1
let pte = page.entry(PTE_V | PTE_R);      // 2
let a = pte.pa();                         // 3
let b = pte.flags();                      // 4
let left = free.len();                    // 5
page.release();                           // 6
let c = page.pa();                        // 7
let d = free.len();                       // 8
```

Two of the numbered lines do not compile. Which two, and why? Why does line 4 compile, when line 3 already passed `pte` to a method that takes `self` by value? Why can `PageGuard` never derive `Copy`?

(c) (5 points) Exercise 03r's `longest` returns the longer of two slices. Does the code below compile? If not, explain what the compiler objects to, given that at run time `r` would point at `a`, the longer slice, which is still alive. Show one way to fix it.

```
pub fn longest<'a>(a: &'a [u8], b: &'a [u8]) -> &'a [u8] {  
    if a.len() >= b.len() { a } else { b }  
}  
  
let a = [1u8, 2, 3];  
let r;  
{  
    let b = [4u8, 5];  
    r = longest(&a, &b);  
}  
println!("{}", r.len());
```

(d) (5 points) Exercise 08r builds a small filesystem. Its directory holds three entries: `README` and `init` are files (`init` contains the 18 bytes `first user process`), and `dev` is a directory. This is its code; the questions are on the next page.

```
pub const NAME_MAX: usize = 14;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum FsError { NotFound, NameTooLong, IsADirectory }

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum InodeKind { File, Dir }

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Inode {
    pub inum: u32,           // the index of this inode's bytes in `data`
    pub kind: InodeKind,
    pub size: usize,
}

pub struct DirEnt {
    pub name: String,
    pub inode: Inode,
}

pub struct FileSystem {
    entries: Vec<DirEnt>,    // the directory: one entry per name
    data: Vec<Vec<u8>>,      // data[i] is the contents of the inode whose inum is i
}

impl FileSystem {
    pub fn find(&self, name: &str) -> Option<Inode> {
        for entry in self.entries.iter() {
            if entry.name == name {
                return Some(entry.inode);
            }
        }
        None
    }

    pub fn lookup(&self, name: &str) -> Result<Inode, FsError> {
        if name.len() > NAME_MAX {
            return Err(FsError::NameTooLong);
        }
        self.find(name).ok_or(FsError::NotFound)
    }

    pub fn read(&self, inode: Inode) -> Result<&[u8], FsError> {
        if inode.kind == InodeKind::Dir {
            return Err(FsError::IsADirectory);
        }
        Ok(&self.data[inode.inum as usize])
    }

    pub fn read_file(&self, name: &str) -> Result<&[u8], FsError> {
        let inode = self.lookup(name)?;
        self.read(inode)
    }
}
```

(d), continued. The filesystem code is on the previous page.

- (i) Rewrite `read_file` without the `?` operator.
- (ii) For each call below, give the value `read_file` returns and say which function produced it. The last name is 16 characters long.

`read_file("init")` `read_file("dev")` `read_file("sixteen_chars_xx")`

2. (_____/20 points)

Process States

Exercise 05r models a process's life as an enum. This is its solution, with the comments shortened. The blank lines are for part (b).

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum ProcState {
    Unused,
    Runnable,
    Running,
    Sleeping { channel: u64 },
    Zombie { exit_status: i32 },
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Event {
    Alloc, // the kernel hands out this table slot
    Schedule, // the scheduler picks this process
    Preempt, // the timer takes the CPU back
    Sleep { channel: u64 }, // block until someone wakes this channel
    Wake { channel: u64 }, // a device or another process signals a channel
    Exit(i32), // the process finishes, with a status
    Reap, // the parent collects the corpse, freeing the slot
}

impl ProcState {
    pub fn can_run(&self) -> bool {
        match self {
            ProcState::Runnable | ProcState::Running => true,
            ProcState::Unused => false,
            ProcState::Sleeping { .. } => false,
            ProcState::Zombie { .. } => false,
        }
    }
}

pub fn next_state(state: ProcState, event: Event) -> Option<ProcState> {
    match (state, event) {
        (ProcState::Unused, Event::Alloc) => Some(ProcState::Runnable),
        (ProcState::Runnable, Event::Schedule) => Some(ProcState::Running),
        (ProcState::Running, Event::Preempt) => Some(ProcState::Runnable),
        (ProcState::Running, Event::Sleep { channel }) => Some(ProcState::Sleeping { channel }),
        (ProcState::Sleeping { channel: c }, Event::Wake { channel: w }) if c == w => {
            Some(ProcState::Runnable)
        }
        (ProcState::Running, Event::Exit(exit_status)) => Some(ProcState::Zombie { exit_status }),
        (ProcState::Zombie { .. }, Event::Reap) => Some(ProcState::Unused),

        _ => None,
    }
}

pub fn apply(state: ProcState, event: Event) -> ProcState {
    match next_state(state, event) {
        Some(new_state) => new_state,
        None => state,
    }
}
```

(a) (6 points) Starting from `ProcState::Unused`, the kernel runs `state = apply(state, e)` for each event `e` below, in order. Give the state after each call, and mark each event that `apply` ignored because `next_state` returned `None`.

Step	Event <code>e</code>	State after the call	Ignored?
1	<code>Alloc</code>		
2	<code>Schedule</code>		
3	<code>Sleep { channel: 0x10 }</code>		
4	<code>Wake { channel: 0x20 }</code>		
5	<code>Schedule</code>		
6	<code>Wake { channel: 0x10 }</code>		
7	<code>Preempt</code>		
8	<code>Schedule</code>		
9	<code>Exit(3)</code>		

(b) (7 points) Add a new event, `Event::Kill`. Killing a process that is `Runnable`, `Running`, or `Sleeping` makes it `Zombie { exit_status: -1 }`. Killing an `Unused` slot or a `Zombie` is illegal. Write the new variant and *one* new match arm in the blank lines of the listing on the previous page. Then explain why it matters where your arm goes relative to `_ => None`, and why its position relative to the other arms does not.

(c) (7 points) Now add a state `ProcState::Stopped` (a process paused by the user, as with Ctrl-Z) and two events: `Stop` moves a `Running` process to `Stopped`, and `Continue` moves a `Stopped` process to `Runnable`.

- (i) You add just the three new variants to the two enums and recompile. Which functions in the listing stop compiling, and which still compile? Explain why, in terms of exhaustiveness. Why is the function that still compiles the more dangerous one?
- (ii) Write the code you must add to finish the change.

3. (_____/20 points)

Scheduling Policies as Traits

Exercise 07r separates the scheduling *policy* from the loop that uses it:

```
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub enum ProcState { Unused, Runnable, Sleeping }

pub trait Scheduler {
    fn pick_next(&mut self, states: &[ProcState]) -> Option<usize>;

    fn run_for(&mut self, states: &[ProcState], turns: usize) -> Vec<usize> {
        let mut picked = Vec::new();
        for _ in 0..turns {
            match self.pick_next(states) {
                Some(slot) => picked.push(slot),
                None => break,
            }
        }
        picked
    }
}

pub struct RoundRobin {
    pub next: usize,
}

impl RoundRobin {
    pub fn new() -> RoundRobin {
        RoundRobin { next: 0 }
    }
}

impl Scheduler for RoundRobin {
    fn pick_next(&mut self, states: &[ProcState]) -> Option<usize> {
        let n = states.len();
        for off in 0..n {
            let slot = (self.next + off) % n;
            if states[slot] == ProcState::Runnable {
                self.next = (slot + 1) % n;
                return Some(slot);
            }
        }
        None
    }
}
```

(a) (6 points) What does `picked` hold after the code below, and what is `rr.next`? Show which slots each call to `pick_next` looks at.

```
use ProcState::*;  
let states = [Sleeping, Runnable, Runnable, Unused, Runnable];  
let mut rr = RoundRobin::new();  
let picked = rr.run_for(&states, 5);
```

(b) (6 points) The 07r file also has a second trait, `Out`, which two types implement, a second `Scheduler`, and two functions that use them:

```
pub trait Out {
    fn write_str(&mut self, s: &str);
    fn write_line(&mut self, s: &str) {
        self.write_str(s);
        self.write_str("\n");
    }
}

impl Out for StringOut { ... } // keeps everything written in a String
impl Out for CountingOut { ... } // only counts the bytes and the calls
impl Scheduler for Priority { ... } // runs the most urgent runnable slot

pub fn write_banner(out: &mut dyn Out) {
    out.write_line("rv6 kernel ready");
}

pub fn trace<S: Scheduler, O: Out>(sched: &mut S, states: &[ProcState],
                                   names: &[&str], turns: usize, out: &mut O) {
    for slot in sched.run_for(states, turns) {
        out.write_line(names[slot]);
    }
}
```

A program calls `trace` four times, with the type pairs $(S, O) = (\text{RoundRobin}, \text{StringOut}), (\text{Priority}, \text{StringOut}), (\text{RoundRobin}, \text{StringOut}),$ and $(\text{Priority}, \text{CountingOut})$. It calls `write_banner` twice, once with a `StringOut` and once with a `CountingOut`.

- (i) How many copies of `trace`, and how many of `write_banner`, does the compiler put in the binary? Name the kind of dispatch each one uses and say how it finds the right method.
- (ii) What goes wrong if you delete `: Scheduler` from `trace`'s type parameters?

(c) (8 points) Write `Backward`, a policy that rotates the other way. Each call scans *downward* from `self.next` (`next`, `next - 1`, ..., `0`, then wrapping around to the last slot), takes the first `Runnable` slot, and leaves the cursor one slot below the one it took (wrapping from `0` to the last slot). Like `RoundRobin`, it must return `None` when nothing is runnable, including for an empty table, and it must never panic. Then give what `Backward::new().run_for(&states, 5)` returns for the table in part (a).

```
pub struct Backward {
    pub next: usize,
}

impl Backward {
    pub fn new() -> Backward {
        Backward { next: 0 }
    }
}

impl Scheduler for Backward {
    fn pick_next(&mut self, states: &[ProcState]) -> Option<usize> {

    }
}
```

4. (_____/20 points)

Streaming with O(1) State

This is `count_fd` from the 12c wc solution. `ulib::read(fd, &mut buf)` reads up to `buf.len()` bytes into `buf` and returns `Ok(n)`, the number it read, which may be fewer than asked for; `Ok(0)` means end of file.

```
#[derive(Default, Clone, Copy)]
struct Counts { lines: usize, words: usize, bytes: usize }

fn is_space(b: u8) -> bool {
    matches!(b, b' ' | b'\t' | b'\n' | b'\r' | 0x0b | 0x0c)
}

fn count_fd(fd: Fd) -> Result<Counts, ulib::Error> {
    let mut c = Counts::default();
    let mut in_word = false;
    let mut buf = [0u8; 512];
    loop {
        let n = ulib::read(fd, &mut buf)?;
        if n == 0 {
            return Ok(c);
        }
        for &b in &buf[..n] {
            c.bytes += 1;
            if b == b'\n' {
                c.lines += 1;
            }
            if is_space(b) {
                in_word = false;
            } else if !in_word {
                in_word = true;
                c.words += 1;
            }
        }
    }
}
```

(a) (6 points) `count_fd(STDIN)` runs, and its calls to `ulib::read` return the bytes below (`␣` is a space, `\n` a newline). Fill in the table with the values after each call has been handled, then give what `count_fd` returns.

Call	read returns	n	c.lines	c.words	c.bytes	in_word
1	ab␣c					
2	d\n␣␣e					
3	f\n					
4	end of file					

`count_fd` returns:

(b) (4 points) A student moves `let mut in_word = false;` from above the `loop` to the first line *inside* it. All five of `wc`'s tests still pass. Their inputs are `one two\nthree\n`, `a b \n`, no newline **here**, the empty input, and two files holding `x\n` and `y z\n`. The test harness answers each `read` with as many of the remaining bytes as fit in the buffer. What does the student's `count_fd` return for the input in part (a)? Why do the tests not catch it?

(c) (10 points) Using `count_fd` as your model, write

```
fn longest_line(fd: Fd) -> Result<usize, ulib::Error>
```

which returns the length in bytes of the longest line read from `fd`, not counting its `\n`. A last line with no trailing newline still counts, and empty input gives 0. For example, `hello\nworld!!\n` gives 7 and `a\n\nbcd` gives 3. Follow `wc`'s rules: one fixed 512-byte buffer, no `Vec` or `String`, and a fixed number of variables however large the input is. Lines can be longer than 512 bytes, and a line can be split across reads.

5. (_____/20 points)

The Commands

The command exercises use these `ulib` calls and `Args` methods:

```
args.len()           // number of arguments, including argv[0], the command's name
args.get(i)          // Option<&[u8]>: argument i, or None if i >= args.len()
ulib::read(fd, &mut buf) // Result<usize, Error>: reads up to buf.len() bytes; Ok(0) = end of file
ulib::write_all(fd, bytes) // Result<(), Error>: writes every byte of bytes
ulib::open(path, O_RDONLY) // Result<Fd, Error>: Err if the file cannot be opened
ulib::close(fd)          // gives the file descriptor back
ulib::write_usize(fd, n, w) // writes n in decimal, right-aligned in w columns
```

(a) (8 points) Many versions of `echo` accept the option `-n`: when the *first* argument is exactly `-n`, `echo` does not print it and does not print the trailing newline. So `echo -n a b` prints `a b` with no newline, `echo -n` prints nothing at all, and `echo a -n` prints `a -n` and a newline. Here is the 10c solution's run. Modify it to support `-n`, using the blank lines, and clearly mark any line you change.

```
fn run(args: Args) -> i32 {

    for i in 1..args.len() {
        if i > 1 {
            let _ = ulib::write_all(STDOUT, b" ");
        }
        let _ = ulib::write_all(STDOUT, args.get(i).unwrap());
    }

    let _ = ulib::write_all(STDOUT, b"\n");

    0
}
```

(b) (6 points) The 11c solution's `cat_fd` is on the left. A student writes the version on the right, with no loop.

```
fn cat_fd(fd: Fd) -> Result<(), ulib::Error> {
    let mut buf = [0u8; 512];
    loop {
        let n = ulib::read(fd, &mut buf)?;
        if n == 0 {
            return Ok(()); // end of file
        }
        ulib::write_all(STDOUT, &buf[..n])?;
    }
}

fn cat_fd(fd: Fd) -> Result<(), ulib::Error> {
    let mut buf = [0u8; 512];
    let n = ulib::read(fd, &mut buf)?;
    ulib::write_all(STDOUT, &buf[..n])?;
    Ok(())
}
```

- (i) `cat`'s six tests check that it copies a file, concatenates several files, reads standard input when given no arguments, reports a missing file, keeps going after a bad file, and copies a 1500-byte file. Every other test file is under 100 bytes, and the test harness answers each `read` with as many of the remaining bytes as fit in the buffer. The student's version passes five of the six tests. Which one fails, and why?
- (ii) With the correct `cat_fd`, how many calls to `ulib::read` and how many to `ulib::write_all` does it take to copy a 1300-byte file, if every `read` returns as many bytes as fit in the buffer?
- (iii) The same file, but now the operating system's `read` never returns more than 128 bytes per call. How many calls of each? Why is a short read like this not an error?

(c) (6 points) This is the rest of the 12c wc solution. Counts, is_space, and count_fd are in Problem 4. The question is on the next page.

```
fn report(c: Counts, label: Option<&[u8]>) {
    let _ = ulib::write_usize(STDOUT, c.lines, 8);
    let _ = ulib::write_usize(STDOUT, c.words, 8);
    let _ = ulib::write_usize(STDOUT, c.bytes, 8);
    if let Some(name) = label {
        let _ = ulib::write_all(STDOUT, b" ");
        let _ = ulib::write_all(STDOUT, name);
    }
    let _ = ulib::write_all(STDOUT, b"\n");
}

fn run(args: Args) -> i32 {
    if args.len() < 2 {
        return match count_fd(STDIN) {
            Ok(c) => {
                report(c, None);
                0
            }
            Err(_) => 1,
        };
    }

    let mut status = 0;
    let mut total = Counts::default();
    let multiple = args.len() > 2;

    for i in 1..args.len() {
        let path = args.get(i).unwrap();
        match ulib::open(path, O_RDONLY) {
            Ok(fd) => {
                match count_fd(fd) {
                    Ok(c) => {
                        report(c, Some(path));
                        total.lines += c.lines;
                        total.words += c.words;
                        total.bytes += c.bytes;
                    }
                    Err(_) => status = 1,
                }
                let _ = ulib::close(fd);
            }
            Err(_) => {
                let _ = ulib::write_all(STDERR, b"wc: cannot open ");
                let _ = ulib::write_all(STDERR, path);
                let _ = ulib::write_all(STDERR, b"\n");
                status = 1;
            }
        }
    }
    if multiple {
        report(total, Some(b"total"));
    }
    status
}
```

(c), continued. `a.txt` holds the 22 bytes `one\two\tthree\n\tfour\n`. `b.txt` holds the 21 bytes `last line, no newline`, with no newline at the end. `nosuch` does not exist. Using the code on the previous page, fill in what this command prints:

```
$ wc a.txt nosuch b.txt
```

	lines	words	bytes	label
stdout, line 1				
stdout, line 2				
stdout, line 3				
stdout, line 4				

Standard error:

Exit status (\$?):

Not every line of the table need be used. Then say how the output changes for `wc b.txt`, and for `wc < b.txt`.

Continue your answers here if necessary.